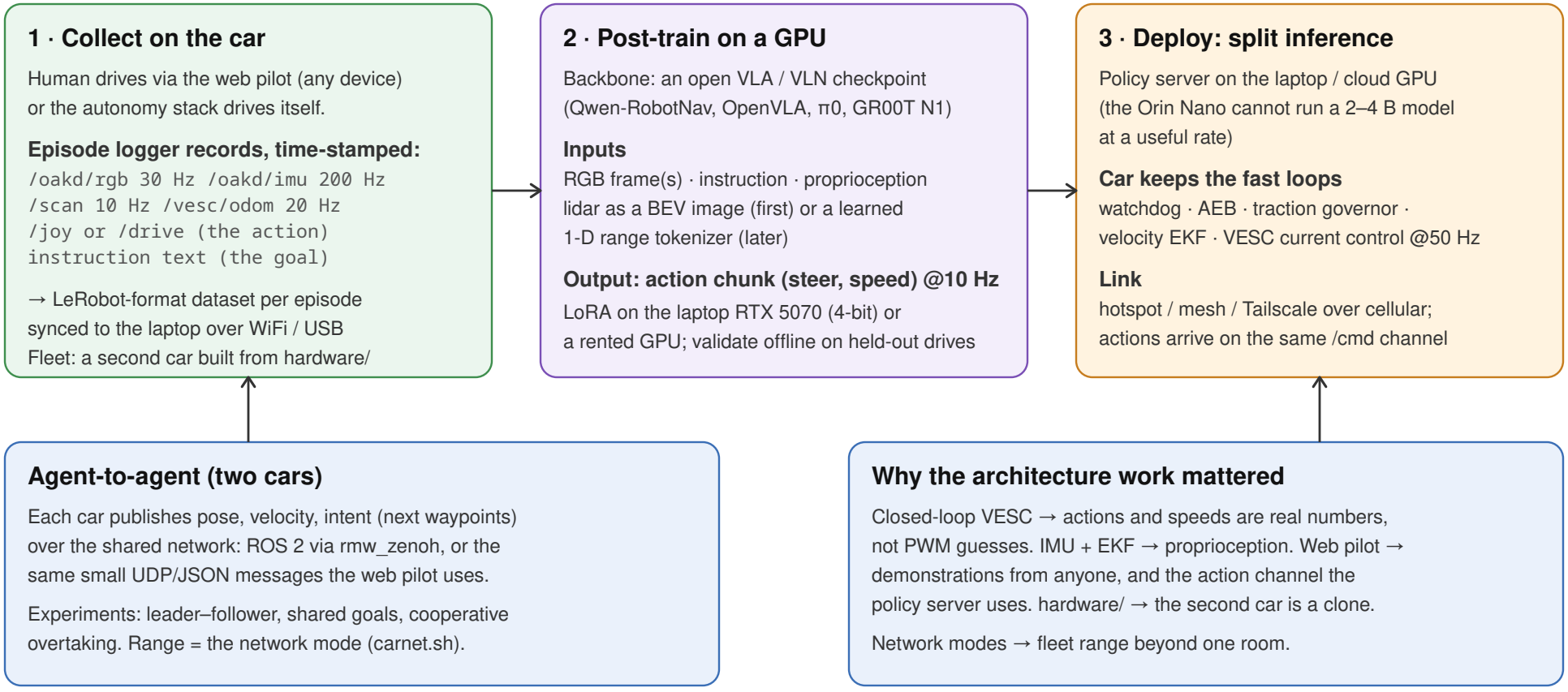
1 · Collect on the car
Human drives via the web pilot (any device)
or the autonomy stack drives itself.
Episode logger records, time-stamped:
/oakd/rgb 30 Hz /oakd/imu 200 Hz
/scan 10 Hz /vesc/odom 20 Hz
/joy or /drive (the action)
instruction text (the goal)
→ LeRobot-format dataset per episode
synced to the laptop over WiFi / USB
Fleet: a second car built from hardware/
2 · Post-train on a GPU
Backbone: an open VLA / VLN checkpoint
(Qwen-RobotNav, OpenVLA, π0, GR00T N1)
Inputs
RGB frame(s) · instruction · proprioception
lidar as a BEV image (first) or a learned
1-D range tokenizer (later)
Output: action chunk (steer, speed) @10 Hz
LoRA on the laptop RTX 5070 (4-bit) or
a rented GPU; validate offline on held-out drives
3 · Deploy: split inference
Policy server on the laptop / cloud GPU
(the Orin Nano cannot run a 2–4 B model
at a useful rate)
Car keeps the fast loops
watchdog · AEB · traction governor ·
velocity EKF · VESC current control @50 Hz
Link
hotspot / mesh / Tailscale over cellular;
actions arrive on the same /cmd channel
Agent-to-agent (two cars)
Each car publishes pose, velocity, intent (next waypoints)
over the shared network: ROS 2 via rmw_zenoh, or the
same small UDP/JSON messages the web pilot uses.
Experiments: leader–follower, shared goals, cooperative
overtaking. Range = the network mode (carnet.sh).
Why the architecture work mattered
Closed-loop VESC → actions and speeds are real numbers,
not PWM guesses. IMU + EKF → proprioception. Web pilot →
demonstrations from anyone, and the action channel the
policy server uses. hardware/ → the second car is a clone.
Network modes → fleet range beyond one room.